轮 趣 科 技

IMU 误差标定教程

推荐关注我们的公众号获取更新资料

版本说明：

版本	日期	内容说明
V1.0	2022/10/31	第一次发布
V1.1	2023/10/19	

网址:www.wheeltec.net

1. IMU 误差标定

首先安装依赖 libdw-dev，在小车系统终端执行以下命令：

```
sudo apt-get install libdw-dev
```

然后需要进行 Ceres 库的安装，首先安装 Ceres 库所需的依赖项。

```
sudo apt-get install liblapack-dev libsuitesparse-dev libcxsparse3
libgflags-dev libgoogle-glog-dev libgtest-dev
```

安装 Ceres 所需依赖之后即可进行 Ceres 的编译安装，我们可以在 https://github.com/ceres-solver/ceres-solver/tree/2.0.0 获取到 Ceres2.0.0 版本的源码亦可直接使用我们提供的源码压缩包。在获取到源码包之后需要在我们的 ubuntu 系统中进行编译安装，在 ubuntu 中解压 Ceres 压缩包后打开终端进入解压后的文件目录下，执行以下操作。

```
mkdir build          #创建 build 目录
cd build             #进入 build 目录
cmake ..             #生成中间文件
sudo make install    #编译安装
```

在安装完 Ceres 之后我们需要获取 code_utils 和 imu_utils 两个功能包，用于 IMU 数据的误差标定分析。这两个功能包我们都有提供 opencv3 和 opencv4 环境下修改后的版本，由于原版在编译时会遇到一些报错，基于这些报错我们对功能包源码进行了一定修改，使得这两个功能包在我们的环境下能直接编译成功。

在编译时需注意，imu_utils 包时依赖于 code_utils 的，所以我们需要先编译 code_utils 功能包，之后再编译 imu_utils。这里我们可以采用单独编译的方式，具体编译指令如下。

```
catkin_make -DCATKIN_WHITELIST_PACKAGES=code_utils
catkin_make -DCATKIN_WHITELIST_PACKAGES=imu_utils
```

编译完成之后，我们可以进行 IMU 数据包的录制。这里需要将 IMU 静置在小车上两小时进行数据包录制。首先是需要开启 IMU 的 ros 节点，用于数据发布，之后利用 rosbag 进行数据包录制，具体指令如下，其中 IMU 话题名及数据包名根据实际情况修改。

```
rosbag record IMU话题名 -O 数据包名.bag
```

两小时之后，在录制终端按下 Ctrl+C 即可完成数据包录制。

打开 imu_utils 功能包中的任意 launch 文件（所有 launch 文件本质是一样的），根据实际情况修改其中的话题名，IMU 名及标定时长三个参数，这里我们录制

的是两小时的数据包，所以将标定时长修改为 120（单位是分钟）。

A3.launch

```
<launch>
    <node pkg="imu_utils" type="imu_an" name="imu_an" output="screen">
        <param name="imu_topic" type="string" value= "/imu"/> <!-- 订阅的imu数据话题名称 -->
        <param name="imu_name" type="string" value= "A3"/> <!-- imu名字，任意设置即可 -->
        <param name="data_save_path" type="string" value= "$(find imu_utils)/data/"/> <!-- 标定结果文件存放位置 -->
        <param name="max_time_min" type="int" value= "120"/> <!-- 标定时长，单位：分钟 -->
        <param name="max_cluster" type="int" value= "100"/> <!-- 默认值 -->
    </node>
</launch>

```

图 1-1-1　IMU 校准调平参考

最后就是进行数据包的回放和 IMU 标定，由于我们录制的数据包时长为 2 小时，按正常速度回放数据包需要花费同等时长，这无疑会消耗许多时间，所以我们可以通过数据包回放加速来减少标定所花费的时间，分别执行以下指令即可完成 IMU 的误差标定。

```
roslaunch imu_utils A3.launch （这里调用刚刚修改过的那个 launch 文件）
rosbag play -r 200  imu_utils/imu.bag （这里要写你录制的包的路径）
```

在误差标定完成之后，会在 imu_utils 功能包下的 data 文件夹中生成标定参数文件，我们需要用到的是 gyr_n、gyr_w、acc_n、acc_w 等三轴平均值。

N100_imu_param.yaml

```
%YAML:1.0
---
type: IMU
name: N100
Gyr:
   unit: " rad/s"
   avg-axis:
      gyr_n: 8.0226069504443656e-04      # Gyro误差模型高斯白噪声
      gyr_w: 4.4153147263889589e-05      # Gyro误差模型随机游走噪声
   x-axis:
      gyr_n: 5.3921043773503836e-04
      gyr_w: 2.2364472813436421e-05
   y-axis:
      gyr_n: 5.8344519509160989e-04
      gyr_w: 2.2667820610429897e-05
   z-axis:
      gyr_n: 1.2841264523066615e-03
      gyr_w: 8.7427148367802434e-05
Acc:
   unit: " m/s^2"
   avg-axis:
      acc_n: 1.0909715156015328e-02      # Acc误差模型高斯白噪声
      acc_w: 4.0347767978459928e-04      # Acc误差模型随机游走噪声
   x-axis:
      acc_n: 1.0761065941039795e-02
      acc_w: 2.6630975523772447e-04
   y-axis:
      acc_n: 8.5197044939375362e-03
      acc_w: 1.6972573784404112e-04
```

图 1-1-2　param.yaml 文件